

Низкоуровневая работа с памятью: Span<T>, Memory<T>, stackalloc

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Екатерина Меттус

.NET разработчик

 @mettus_katerina

 mettus.ekaterina@gmail.com

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в чате учебной группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Span<T>

Memory<T>

stackalloc

Рефлексия

Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

1. Применять `Span<T>`

2. Применять `Memory<T>`

3. Использовать `stackalloc`

Обработка данных без лишних
аллокаций и копирования



Span<T>

Зачем?

Работа с непрерывными блоками данных с помощью массивов и строк.

Необходимость аллокаций: при создании или копировании подмассивов или подстрок происходят дополнительные аллокации в управляемой куче.

```
string csv = "Alice,25,London";  
string[] parts = csv.Split(',');
```

```
var span = line.AsSpan();  
var firstComma = span.IndexOf(',');  
var idSpan = span.Slice(0, firstComma);
```



Зачем?

Method	CharactersCount	Mean	StdDev	Scaled	Gen 0	Allocated
Substring	10	8.277 ns	0.1938 ns	4.54	0.0191	40 B
Slice	10	1.822 ns	0.0383 ns	1.00	-	0 B
Substring	1000	85.518 ns	1.3474 ns	47.22	0.4919	1032 B
Slice	1000	1.811 ns	0.0205 ns	1.00	-	0 B

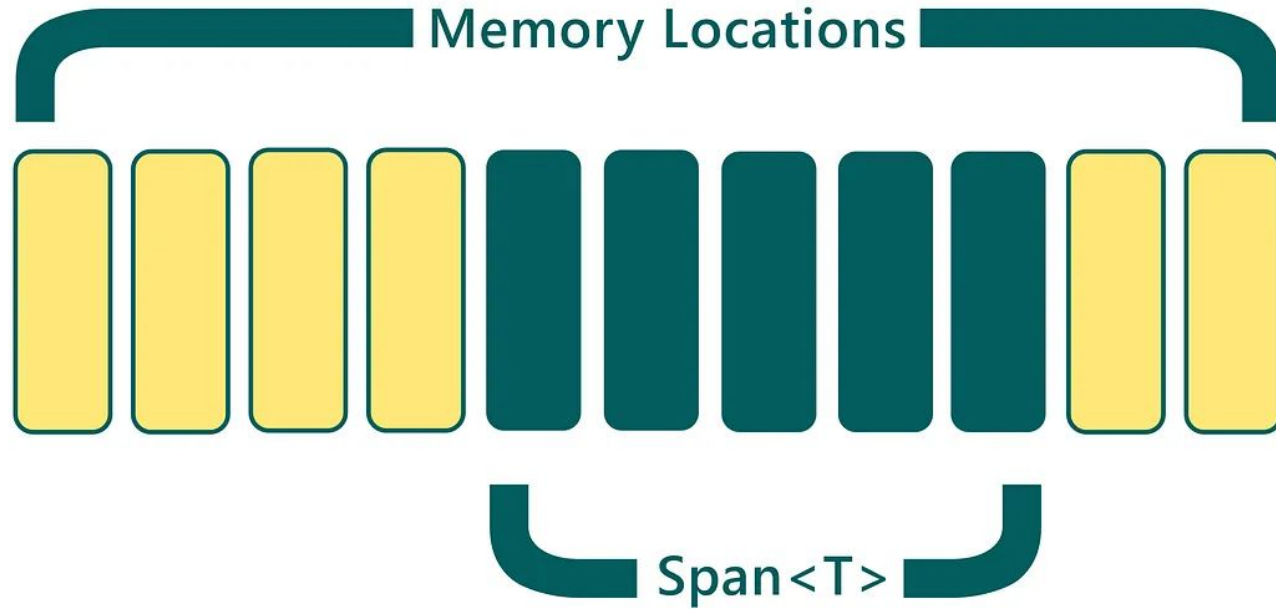
Span<T>

Основная идея: предоставить безопасный, максимально быстрый и zero-allocation способ работы с подпоследовательностями без риска для безопасности и избыточных аллокаций. Благодаря этому, можно обрабатывать даже гигантские массивы данных, выделяя из них срезы без копирования.

```
int[] arr = new int[] { 10, 11, 12, 13, 14 };  
Span<int> s = arr.AsSpan(1, 3); // Ссылается на элементы arr[1]..arr[3]  
s[0] = 42; // arr[1] теперь равен 42!
```



Span<T>



Span<T>

Упрощение API

```
int Parse(string input);  
int Parse(string input, int startIndex, int length);  
unsafe int Parse(char* input, int length);  
unsafe int Parse(char* input, int startIndex, int length);
```

```
int Parse(Span<char> input);
```



Особенности

- Определен индексатор, возвращающий значение по ссылке, поэтому его можно использовать как массив
- Можно эффективно использовать поддиапазоны
- Для того чтобы блокировать «опасные» сценарии размещения в куче, передачи между потоками и неопределенного времени жизни, `Span<T>` реализован как `ref struct`

Особенности

- Не может представлять элемент массива
- Не может представлять поле класса или обычной структуры
- Не может быть упакован в `System.ValueType` или `System.Object`
- Нельзя использовать в лямбдах и анонимных методах
- Нельзя использовать с `async/await`, `yield return`

ReadOnlySpan<T>

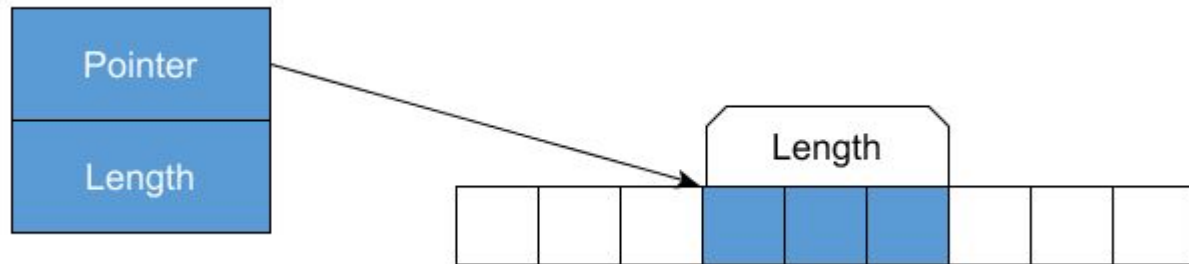
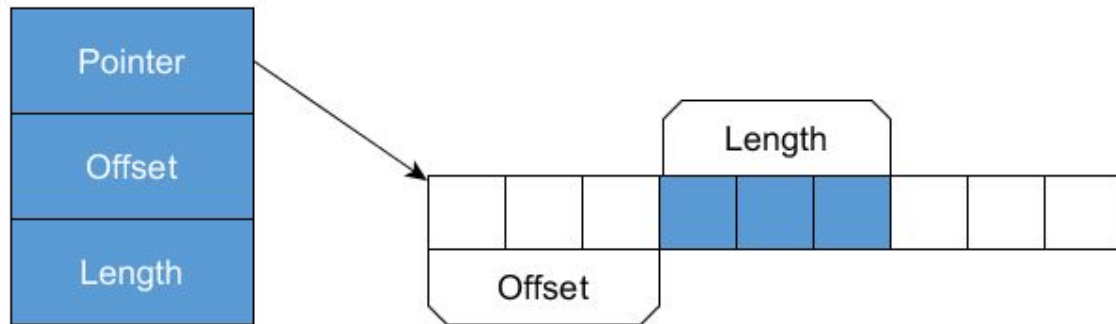
Представляет память, которую нельзя изменить

Стандартное применение - строки

```
ReadOnlySpan<char> readOnlySpan = "This is a sample data for  
testing purposes.";
int index = readOnlySpan.IndexOf(' ');
var data = ((index < 0) ?
    readOnlySpan : readOnlySpan.Slice(0, index)).ToArray();
```



Внутреннее устройство



Внутреннее устройство

```
public readonly ref struct Span<T>
{
    internal readonly
    ByReference<T> _pointer;
    private readonly int _length;
    //Other members
}
```



Основные операции

 Операция

 Описание

 Аллокации

`span[index]`

Индексация: доступ к элементу по индексу

 Нет

`Slice()`

Возврат подотрезка (view на ту же память)

 Нет

`CopyTo()`

Копирование в другой `Span<T>` или массив

 Нет

`ToArray()`

Копирование в новый массив

 Да

`Fill()`

Заполнение всех элементов заданным значением

 Нет

`Clear()`

Обнуление всех элементов

 Нет

`foreach/for`

Перебор элементов

 Нет

`Reverse()`

Инвертирование порядка элементов in-place

 Нет

Slice()

Один из фундаментальных методов `Span<T>` — это `.Slice(int start, int length)`, который возвращает новый `Span`, ссылающийся на подотрезок исходных данных, не затрагивая память и не производя новых аллокаций.

Пример использования `.Slice`:

```
Span<int> data = stackalloc int[10];  
Span<int> sub = data.Slice(2, 5); // элементы с 2-го по 6-й
```

В отличие от массивов, где пришлось бы копировать диапазон, `Slice` лишь "сдвигает" указатель `_pointer` и меняет значение `_length` — мгновенно и без копирования или аллокации. Такой подход абсолютно zero-allocation и чрезвычайно быстр.

```
Span<int> arr = stackalloc int[5];  
var fail = arr.Slice(4, 2); // ошибка: длина 2 выходит за пределы массива
```



Использование

Парсинг строк без лишних аллокаций

Парсинг строк — один из классических сценариев, где Span позволяет добиться максимальной эффективности и полного zero-allocation, особенно если нужно обрабатывать большие журналы, логи, данные CSV или протоколы.

```
string input = "apple,42,blue";
ReadOnlySpan<char> span = input.AsSpan();

int index1 = span.IndexOf(',');
ReadOnlySpan<char> first = span.Slice(0, index1);

span = span.Slice(index1 + 1);
int index2 = span.IndexOf(',');
ReadOnlySpan<char> second = span.Slice(0, index2);

ReadOnlySpan<char> third = span.Slice(index2 + 1);
```



Работа с сетевыми буферами и неуправляемой памятью

Сетевые приложения и низкоуровневые системы очень часто манипулируют массивами байт при приёме или отправке данных по сокетам. Span идеально подходит для этих задач: позволяет создать удобные окна поверх буфера без копирования данных.

```
byte[] buffer = new byte[1024];  
int bytesRead = socket.Receive(buffer);  
Span<byte> packet = buffer.AsSpan(0, bytesRead);  
  
// Допустим, первые 4 байта — это int32  
int header = BitConverter.ToInt32(packet.Slice(0, 4));  
Span<byte> payload = packet.Slice(4);  
  
// Можно парсить данные payload далее...
```





LIVE



Memory<T>

Memory<T>

Основная идея: Много ограничений Span → Появилась необходимость универсального, безопасного и zero-allocation механизма передачи буферов, совместимого с любой областью памяти, с возможностью хранения в классе, передачи между потоками/асинхронными задачами и интеграции с существующими API..

```
byte[] array = new byte[] {1, 2, 3};  
Memory<byte> memory1 = new Memory<byte>(array);  
Memory<byte> memory2 = new Memory<byte>(array, start: 1, length: 2);  
ReadOnlyMemory<char> memory3 = "Hello world".AsMemory();
```



Особенности

Блок памяти, который можно спокойно передавать методам и получать от методов

Типом `Memory<T>` можно обрабатывать следующие данные:

- массив `T[]` – играет роль заранее выделенного буфера, повторно используемого на протяжении асинхронного вызова или в API, не допускающих использования `Span<T>`;
- `string` – в этом случае она представлена типом `ReadOnlyMemory<char>`;
- любой тип, реализующий интерфейс `IMemoryOwner<T>`



Особенности

Использовать можно следующими способами:

- для локального эффективного применения из него можно сгенерировать `Span<T>`;
- по `Memory<char>` можно сгенерировать строку, вызвав метод `ToString`, в остальных случаях можно использовать метод `ToArray` (оба метода выделяют память для нового ссылочного типа!);
- как и для `Span<T>`, можно вырезать части различными методами `Slice`.

Внутреннее устройство

Внутреннее устройство `Memory<T>`:

- `_object` — ссылка на массив, строку или пользовательский пул (тип `object`).
- `_index` — смещение в исходном массиве/буфере.
- `_length` — длина сегмента.

Конструкторы `Memory<T>`:

`Memory<T> memory = new Memory<T>(array);` // Ссылка на весь массив

`Memory<T> memory = new Memory<T>(array, start, length);` // Срез массива

Основные операции

 Операция	 Описание	 Аллокации
<code>memory[index]</code>	Индексация через <code>.Span[index]</code>	 Нет
<code>Slice()</code>	Возврат подотрезка <code>Memory<T></code>	 Нет
<code>CopyTo()</code>	Копирование в другой <code>Memory<T></code> или <code>Span<T></code>	 Нет
<code>ToArray()</code>	Копирование в новый массив	 Да
<code>Fill()</code>	Заполнение значением через <code>.Span.Fill()</code>	 Нет
<code>Clear()</code>	Обнуление памяти через <code>.Span.Clear()</code>	 Нет
<code>foreach/for</code>	Перебор элементов через <code>.Span</code>	 Нет
<code>Pin()</code>	Закрепление памяти для небезопасного доступа (<code>ref T*</code>)	 Нет
<code>Retain()</code>	Увеличение счётчика владения (в <code>IMemoryOwner<T></code>)	 Нет

Slice()

Метод Slice — один из ключевых для доступа к поддиапазону памяти внутри любого буфера. В Memory<T> он реализован следующим образом:

```
public Memory<T> Slice(int start)
public Memory<T> Slice(int start, int length)
```

Вызов этого метода возвращает новый объект Memory<T>, указывающий на поддиапазон исходного буфера, не создавая ни одной новой аллокации массива:

```
Memory<byte> memory = new Memory<byte>(array, 10, 100);
Memory<byte> slice = memory.Slice(30, 20); // Подмножество памяти
```

Внутренне происходит создание нового объекта, который хранит тот же _object, но измененные _index и _length — с учетом сдвига и длины.



Использование

Асинхронное чтение из Stream

Теперь I/O-методы потоков в .NET стандартной библиотеки, начиная с .NET Core 2.1, принимают параметры типа `Memory<T>`, что позволяет эффективно читать данные в произвольные буферы без аллокаций:

```
public async Task ReadFileAsync(Stream stream)
{
    byte[] buffer = new byte[4096];
    Memory<byte> memory = new Memory<byte>(buffer);
    int bytesRead = await stream.ReadAsync(memory);
    // Дальнейшая обработка buffer с учетом bytesRead
}
```



Хранение буфера Memory в поле класса

Благодаря своей структуре, Memory отлично решает задачу долгоживущего буфера, который может быть сохранён в классе, передан в другие методы, разделён на срезы и быть легко управляемым при масштабировании приложения.

```
public class BufferManager
{
    private Memory<byte> _buffer;

    public BufferManager(int size)
    {
        _buffer = new byte[size];
    }

    public Memory<byte> GetSlice(int start, int
length)
    {
        return _buffer.Slice(start, length);
    }
}
```



Использование

Парсинг данных

Очень часто задачу парсинга данных выгоднее всего решать с помощью `Span<T>`, получаемого из `Memory<T>`, что позволяет писать максимально быстрые и zero-allocation парсеры.

```
public void ParseBuffer(Memory<byte> memory)
{
    Span<byte> span = memory.Span;
    int value = BitConverter.ToInt32(span.Slice(0, 4));
    // Обработка остальных данных
}
```



Рекомендации

- Используйте `Span<T>` или `Memory<T>` в специализированном высокопроизводительном коде – как правило, включать их в код бизнес-приложений не имеет смысла.
- Отдавайте предпочтение передаче `Span<T>` в качестве аргумента метода, если это возможно, – он быстрее работает (благодаря поддержке со стороны среды выполнения) и может представить больше видов памяти. Но в асинхронном коде альтернативы `Memory<T>` не существует.
- Отдавайте предпочтение варианту, допускающему только чтение, чтобы сделать код безопаснее. Не используйте по умолчанию вариант, допускающий изменение. Версия только для чтения более «либеральна»; например, если метод принимает `Span<T>`, то передать ему аргумент типа `ReadOnlySpan<T>` не получится, а если он принимает `ReadOnlySpan<T>`, то можно передать и `Span<T>`, и `ReadOnlySpan<T>`.





LIVE



stackalloc

stackalloc

Управляемый, но при этом эффективный и контролируемый способ временного выделения памяти

```
unsafe
{
    int* numbers = stackalloc int[5];
    for (int i = 0; i < 5; i++)
    {
        numbers[i] = i * 10;
    }
    // numbers автоматически освобождается при выходе из блока
}
```



Работа с Span

С C# 7.2 для использования `stackalloc` не требуется небезопасный код, если задействовать `Span<T>`:

```
Span<byte> buffer = stackalloc byte[16];  
buffer[0] = 42;
```

Начиная с C# 8.0, появилась возможность инициализировать буфер непосредственно в выражении:

```
Span<int> values = stackalloc int[] { 1, 2, 3, 4, 5 };
```





Характеристика



Подробности

Только локальное использование

Буфер живёт до конца текущей области видимости (обычно — до завершения метода)

Нельзя использовать с `Heap`-типами

Допустимы только `unmanaged`-типы: примитивы и структуры без ссылок

Размер — константа или вычисляемое выражение

Размер должен быть известен на момент выполнения, но не превышать лимит стека

Не поддерживается выделение на куче

`stackalloc` всегда выделяет память на стеке текущего потока

Использование с `Memory<T>` запрещено

Нельзя обернуть `stackalloc`-буфер в `Memory<T>` — невозможно гарантировать lifetime

Ограничение по размеру

Явного лимита нет, но стек ограничен (~1–2 МБ); большие буферы → `StackOverflow`

Нельзя использовать вне метода

`stackalloc` допустим только в теле метода; нельзя использовать в полях классов

Нельзя передать буфер за пределы стека

Нельзя возвращать `Span<T>`, основанный на `stackalloc`, из метода — он станет недействительным



Вопросы



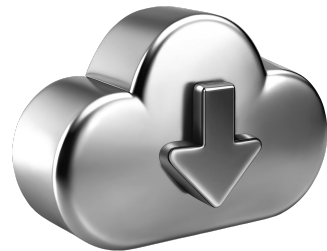
если есть вопросы



если вопросов нет

Список материалов для изучения

1. Konrad Kokosa “Pro .NET Memory Management”
2. Ben Watson “Writing High-Performance .NET Code”
3. [Writing High-Performance Code Using Span<T> and Memory<T> in C#](#)
4. [Span – Adam Sitnik – .NET Performance and Reliability](#)



**Подведём
итоги занятия**

Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

1. Применять `Span<T>`

2. Применять `Memory<T>`

3. Использовать `stackalloc`

Обработка данных без лишних
аллокаций и копирования



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️